A Reproducible Data Workflow for the Titanic Passenger Dataset

Mohammad AfsharNejad

Master's Degree in Artificial Intelligence, Udacity and Woolf University

Master's Degree in AI Capstone: AI Programming Foundations Project

October 20, 2026

A Reproducible Data Workflow for the Titanic Passenger Dataset

Overview

I built a reproducible data workflow in a Jupyter notebook that loads, cleans, explores and visualizes passenger data from the Titanic disaster. The dataset is Titanic - Machine Learning from Disaster, published on Kaggle (Cukierski, 2012) and available at https://www.kaggle.com/c/titanic/data. The notebook uses documented Python functions for cleaning and exploratory analysis and saves three labeled figures, so the cleaned table can serve as a starting point for later machine learning projects.

Dataset Description

The dataset is the training file of the Kaggle Titanic competition, and each row describes one passenger of the RMS Titanic (Cukierski, 2012). It has 891 rows and 12 columns. The target variable is Survived (0 = died, 1 = survived), and 38.4% of the passengers in the file survived. The main explanatory variables are ticket class (Pclass), Sex, Age, the number of siblings or spouses aboard (SibSp), the number of parents or children aboard (Parch), Fare and the port of embarkation (Embarked), while PassengerId, Name, Ticket and Cabin are identifiers or free text. Three columns have missing values: Age (177 rows, 19.9%), Cabin (687 rows, 77.1%) and Embarked (2 rows, 0.2%).

Workflow Description

The workflow follows the usual order of a data science project, in which data are first preprocessed and then described with exploratory analysis and visualization before any modeling takes place (Danchev, 2022). In my notebook, each of these steps has its own Markdown section: Imports & Loading, Data Cleaning, Exploratory Analysis, Visualizations, and Summary & Interpretation.

Ingestion. The notebook reads titanic.csv with pandas through a relative path and stores it as raw_df. It then shows the first five rows, the shape, the data types and a missing-value table produced by the helper function report_missing.

Cleaning. Two cleaning functions, handle_missing_values and standardize_and_engineer, each work on a copy of their input. Together they produce clean_df, which has 891 rows, 14 columns and no missing values.

Exploratory analysis. The function explore returns summary statistics, survival rates by group and a correlation matrix in one dictionary. I reused the same function on subgroups to check survival for imputed ages, recorded cabins, family sizes and age bands.

Visualization and summary. A helper function, save_figure, writes each of the three figures to the figures folder as a 150-dpi PNG file. The notebook ends with a Markdown summary of the findings, assumptions and limitations.

Key Decisions and Assumptions

Keeping the Raw Data Unchanged

Wilson et al. (2017) recommend keeping raw data exactly as it was obtained instead of overwriting it with cleaned versions. For this reason, raw_df is never modified, and every cleaning function starts by copying its input. The notebook confirms after cleaning that raw_df still contains its 177 missing ages, so the original data are always available for comparison.

Filling Missing Values Instead of Dropping Rows

Analyzing only complete cases can bias results when the chance of a value being missing depends on other variables, and it also lowers precision because the sample becomes smaller (Sterne et al., 2009). In this dataset, dropping rows with a missing age would remove 177 passengers (19.9%). The missing ages are also unevenly spread: 27.7% of third-class passengers

have no recorded age, compared with 13.9% in first class and 6.0% in second class. Dropping these rows would therefore under-represent third-class passengers, who had the lowest survival rate.

Age: Median by Class and Sex

The function handle_missing_values fills each missing age with the median age of passengers with the same Pclass and Sex. I used groups because the median ages differ strongly, from 21.5 years for third-class women to 40.0 years for first-class men, so one overall median would make first-class passengers look too young and third-class passengers too old. I used the median rather than the mean so that a few older passengers (up to 80 years) do not pull the fill value upward. The function also adds an AgeImputed flag, so the 177 filled rows can be identified in every later step.

Cabin: Replaced by a Flag

Cabin is missing for 77.1% of passengers, so filling it with a guessed cabin number would add mostly invented values. The missing values are also strongly linked to class: Cabin is missing for 97.6% of third-class passengers but only 18.5% of first-class passengers. I therefore replaced the text column with a HasCabin flag (1 = cabin recorded), which keeps the information that a cabin number exists without inventing any cabin numbers.

Embarked: Most Frequent Port

Only two rows are missing a port of embarkation. I filled them with the most common port, Southampton (S), because such a small change cannot noticeably affect the group results.

Data Types, Duplicates and Family Size

The function standardize_and_engineer converts Sex, Embarked and Pclass to the categorical type, because they hold a small fixed set of labels, and Pclass is a class label rather

than a measured quantity. It also checks for exact duplicate rows; none were found, and the check is printed so that the result is documented. Finally, it adds FamilySize (SibSp + Parch + 1), which combines two related columns into one variable that is easier to interpret.

Focus of the Exploratory Analysis

The exploratory analysis focused on survival, the target variable for later modeling. I grouped the survival rate by Sex, Pclass and Embarked, and I added checks for the two new flags because they show whether the missing values themselves are related to survival. A correlation matrix gave an overview of linear relationships between all numeric variables.

Purpose of Each Figure

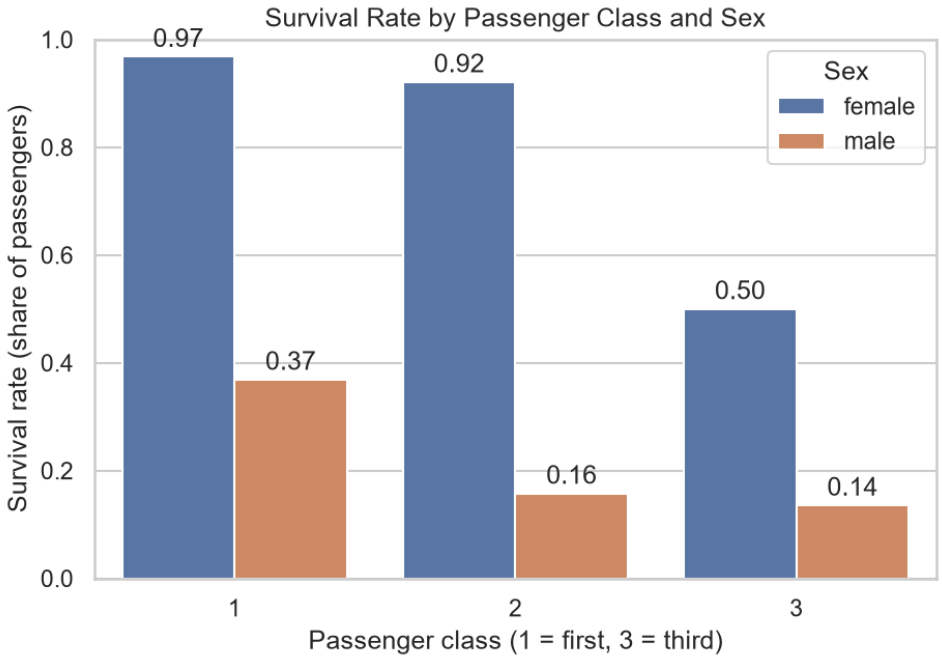
Figure 1 was designed to show how the two strongest group factors, sex and class, combine. Figure 2 was designed to compare the age distributions of survivors and non-survivors; it uses only the 714 passengers with a recorded age, so the six median fill values do not create artificial peaks. Each group in Figure 2 is scaled to its own density, so the shapes can be compared even though the groups have different sizes. Figure 3 was designed to give one overview of all pairwise linear relationships, including the new features HasCabin, AgeImputed and FamilySize.

Results and Interpretation

The overall survival rate is 38.4%. Women survived at 74.2% and men at 18.9%. Survival also fell with class, from 63.0% in first class to 47.3% in second class and 24.2% in third class.

Figure 1

Survival Rate by Passenger Class and Sex

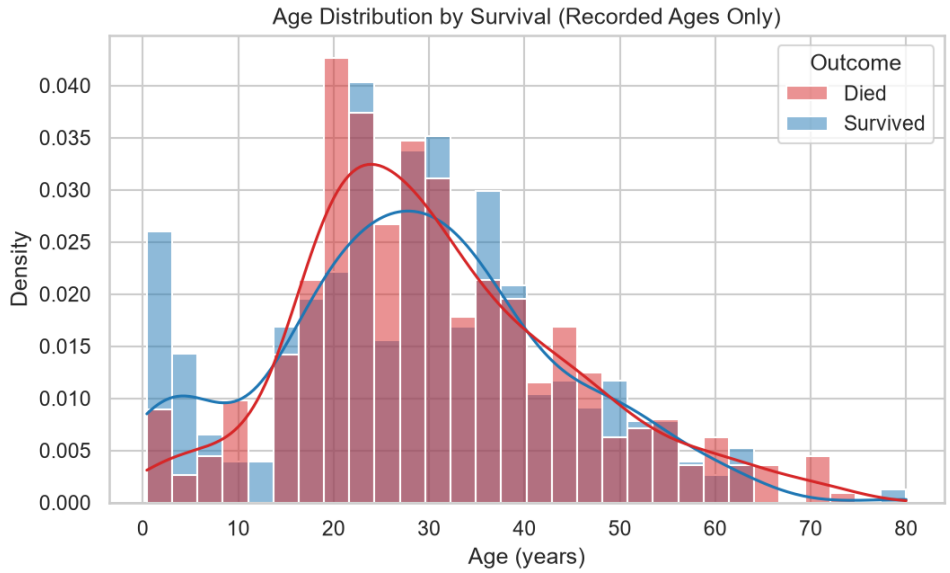


Note. Cleaned data, n = 891. Bar labels show the share of passengers in each group who survived.

Figure 1 shows that sex and class combine strongly. In every class, women survived at a much higher rate than men. Almost all first-class women (96.8%) and second-class women (92.1%) survived, but only 50.0% of third-class women did. For men, the largest drop is between first class (36.9%) and second class (15.7%), and third-class men had the lowest survival rate of any group (13.5%). Class therefore matters for both sexes, but in a different place for each.

Figure 2

Age Distribution of Survivors and Non-Survivors (Recorded Ages Only)

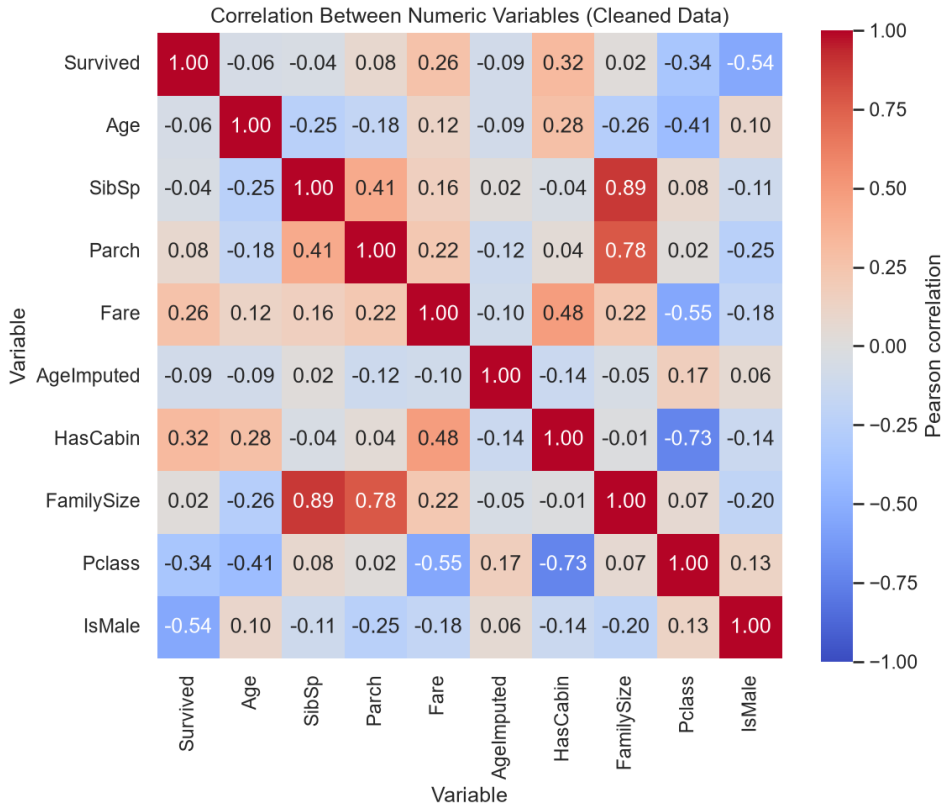


Note. Only the 714 passengers with a recorded age are shown. Each group is scaled to its own density.

Figure 2 shows that the two age distributions overlap for most ages. The clearest difference is among young children: the survivor curve has a peak below about 10 years that the non-survivor curve does not have. The age-band table in the notebook confirms this, with a survival rate of 61.3% for children under 10 (62 passengers), compared with 35.1% for passengers aged 18 to 29 and 41.8% for those aged 30 to 49. Because the distributions overlap so much outside early childhood, the linear correlation between Age and Survived is weak (-0.06).

Figure 3

Correlation Matrix of Numeric Variables After Cleaning



Note. Pearson correlations. Pclass is coded 1 to 3, and IsMale is 1 for male passengers.

Figure 3 shows that being male has the strongest correlation with survival (-0.54), followed by Pclass (-0.34), HasCabin (0.32) and Fare (0.26). HasCabin, Fare and Pclass are also strongly related to each other (for example, -0.73 between HasCabin and Pclass and -0.55 between Fare and Pclass). This suggests that they carry overlapping information about a passenger's class and wealth rather than three separate effects. FamilySize has almost no linear correlation with survival (0.02).

Summary of Insights, Assumptions and Limitations

The main insight is that sex and class explain most of the survival differences in this sample, and that the cleaning flags carry information of their own. Passengers with a recorded

cabin survived at 66.7% compared with 30.0% for those without one, and passengers with a recorded age survived at 40.6% compared with 29.4% for those whose age was imputed. A surprising pattern is that FamilySize shows no linear correlation with survival, yet the grouped table shows passengers travelling alone survived at 30.4%, families of two to four at 55.3% to 72.4%, and most larger families at lower rates. Passengers from Cherbourg also survived more often (55.4%) than those from Southampton (33.9%), but I did not test whether this is explained by class mix, so I do not treat the port as a cause. The results depend on two assumptions: that missing ages are similar to recorded ages within the same class and sex, and that a missing cabin number mainly means that no cabin was recorded. All results describe this sample of 891 passengers and are associations, not causal effects.

Responsible Practice (Bias and Data Quality)

Filling a missing value with a single number treats it as if it had been observed, which ignores the uncertainty about the true value and makes variability and standard errors look smaller than they are (Sterne et al., 2009). This happened in my data: the standard deviation of Age fell from 14.53 years for the recorded ages to 13.30 years after imputation, and all 177 imputed passengers share only six possible ages. To reduce this risk, I kept the AgeImputed flag and built Figure 2 from recorded ages only. In later work, I would compare results with and without the imputed rows, or use multiple imputation, before drawing conclusions about age.

The missing values in this dataset are not random. Whether data are missing at random given the observed variables or missing for reasons that were not recorded cannot be decided from the observed data alone (Sterne et al., 2009). My group-median method uses class and sex, which are linked to missingness, but it still assumes that within each group the passengers without an age are like those with one. The lower survival rate among passengers with an

imputed age (29.4% compared with 40.6%) shows that these passengers differ in at least one important way, partly because more of them were in third class. Any model that uses the filled ages could therefore carry this hidden difference.

The HasCabin flag reflects how records were kept as much as it reflects the passengers. Because cabin numbers are missing for almost all third-class passengers, a model could treat HasCabin as a separate effect when it mostly repeats class information. I would check models with and without this flag and report the overlap with Pclass and Fare shown in Figure 3.

The data are a historical sample of 891 passengers from one event in 1912, and Sex is recorded only as male or female. The patterns reflect the social rules and class structure of that voyage, so they should not be generalized to modern settings or used to judge groups of people today. I report the results as descriptions of this sample and avoid causal language.

Reproducibility

Anyone can rerun the project by cloning the repository, installing requirements.txt and running data_workflow.ipynb. The packages were installed in a dedicated Python 3.11 virtual environment, and requirements.txt was created from that environment with pip freeze, so it pins exact versions such as pandas 3.0.6, NumPy 2.4.6, Matplotlib 3.11.2 and seaborn 0.13.2. Danchev (2022) explains that a requirements.txt file listing every package and version lets others rebuild the same computing environment and reduces errors caused by later package updates. This is why I kept the tools for building this report in a separate environment, so requirements.txt lists only what the notebook needs.

The notebook uses relative paths only (titanic.csv and figures/), so it runs on any computer from the project folder. I executed it from top to bottom with jupyter nbconvert --execute and saved it with all outputs, and it ran without errors. Danchev (2022) also notes that a

notebook can combine code, outputs and narrative text in one document, which makes the analysis process transparent. I used this by placing Markdown explanations, figure captions and the final summary next to the code that produced them.

Wilson et al. (2017) recommend tracking work with version control in small, frequent changes, listing dependencies explicitly and adding a short README file at the project root. I followed this with Git: the repository has a main branch and a dev branch, I committed after each task with a clear message, and I merged dev into main at the end. The README explains how to install the requirements and run data_workflow.ipynb, and a .gitignore file keeps the virtual environments out of the repository.

Future Integration Reflections

If this data were used to train a machine learning model, I would split it into training and test sets before cleaning and fit the group medians, the Embarked mode and the encoders on the training set only, inside a pipeline, because preprocessing fitted on all the data leaks test information and gives over-optimistic scores (scikit-learn developers, 2026). I would also one-hot encode Sex, Embarked and Pclass, drop identifier columns such as PassengerId, Name and Ticket, and keep the AgeImputed and HasCabin flags as features. Before training a neural network, I would scale Age, Fare and FamilySize (Fare ranges from 0 to 512.33 with a median of 14.45, so a log transform would help), check the class balance of 38.4% survivors to 61.6% non-survivors, and keep in mind that 891 rows is small for a neural network, so cross-validation and strong regularization would be needed. Agentic automation could help by rerunning the notebook with nbconvert after each change, checking that the missing-value counts and figures are unchanged, and drafting commit messages. It would be risky if an agent changed cleaning rules on its own, for example by dropping the 177 rows with missing ages, because that would

shift the class mix and the results in Figures 1 to 3 without anyone noticing, so I would keep a human review before every commit.

Sources and Citations

References

Cukierski, W. (2012). *Titanic - Machine learning from disaster* [Data set]. Kaggle. https://www.kaggle.com/c/titanic/data

Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education, 5*(56), Article 156. https://doi.org/10.21105/jose.00156

scikit-learn developers. (2026). *Common pitfalls and recommended practices* (Version 1.9.1). scikit-learn. https://scikit-learn.org/stable/common_pitfalls.html

Sterne, J. A. C., White, I. R., Carlin, J. B., Spratt, M., Royston, P., Kenward, M. G., Wood, A. M., & Carpenter, J. R. (2009). Multiple imputation for missing data in epidemiological and clinical research: Potential and pitfalls. *BMJ, 338*, Article b2393. https://doi.org/10.1136/bmj.b2393

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology, 13*(6), Article e1005510. https://doi.org/10.1371/journal.pcbi.1005510